Jeevan Bank API Documentation

Version 1.0 | For Frontend Developers

Table of Contents

---

1. Introduction

1.1 Overview

Jeevan Bank is a Spring Boot REST API that provides comprehensive banking functionality including:

- User registration and authentication
- Account management (open, view, update, close, delete)
- Financial transactions (deposit, withdraw, transfer)
- Loan applications and management
- Debit/Credit card management
- Cheque book requests

1.2 Technology Stack

Component	Technology
Framework	Spring Boot 3.2.2
Language	Java 17+
Database	PostgreSQL
Authentication	JWT (JSON Web Tokens)
Security	Spring Security + BCrypt

1.3 Base Configuration

Setting	Value
Base URL	`http://localhost:8080`
Content-Type	`application/json`
CORS Origins	`http://localhost:3000`, `http://localhost:5173`

1.4 Response Format

All API responses follow a consistent wrapper format:

```
{
  "success": true,
  "message": "Operation successful",
  "data": { ... }
}
```

Example Success Response:

```
{
  "success": true,
  "message": "Account opened successfully",
  "data": {
    "accountId": "550e8400-e29b-41d4-a716-446655440000",
    "accountNumber": "JB123456789012345678",
    "accountType": "SAVINGS",
    "balance": 0.00,
    "status": "Active"
  }
}
```

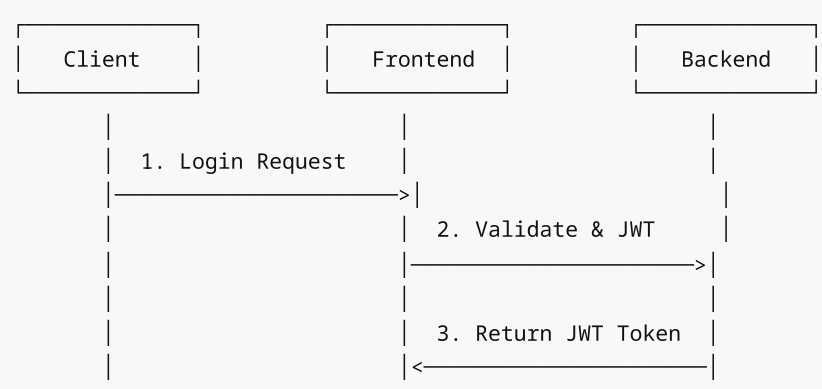
Example Error Response:

```
{
  "success": false,
  "message": "Insufficient balance",
  "data": null
}
```

---

2. Authentication

2.1 Authentication Flow



```
        |                          |                          |
        |  4. Store Token          |                          |
        |<-------------------------|                          |
        |                          |                          |
        |  5. Authenticated        |                          |
        |  Requests with           |                          |
        |  Bearer Token            |                          |
        |------------------------->|------------------------->|
```

2.2 User Roles

Role	Description	Access Level
`ROLE_USER`	Standard bank customer	Own accounts only
`ROLE_ADMIN`	Bank administrator	All accounts and operations

2.3 JWT Token Usage

After successful login, include the JWT token in the `Authorization` header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

---

3. API Endpoints

3.1 Authentication Endpoints (Public)

POST /api/auth/register

Register a new user account.

Request:

```json
{
  "username": "johndoe",
  "password": "SecurePass123!",
  "email": "john@example.com",
  "firstName": "John",
  "lastName": "Doe"
}
```

Response (201 Created):

```json
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "userId": "uuid",
    "username": "johndoe",
```

```
    "email": "john@example.com",
    "role": "ROLE_USER"
  }
}
```

POST /api/auth/login

Authenticate user and receive JWT token.

Request:

```
{
  "username": "johndoe",
  "password": "SecurePass123!"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiJ9...",
    "userId": "uuid",
    "username": "johndoe",
    "email": "john@example.com",
    "role": "ROLE_USER"
  }
}
```

3.2 User Account Endpoints

POST /api/user/openaccount

Open a new bank account.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "dateOfBirth": "1990-05-15",
  "address": "123 Main Street, City, State 12345",
  "phone": "+1234567890",
  "citizenshipId": "US123456789",
  "accountType": "SAVINGS"
}
```

Account Types: `SAVINGS`, `CHECKING`, `FIXED_DEPOSIT`

Response (201 Created):

```
{
  "success": true,
  "message": "Account opened successfully",
  "data": {
    "accountId": "uuid",
    "accountNumber": "JB123456789012345678",
    "accountType": "SAVINGS",
    "balance": 0.00,
    "status": "Active",
    "createdAt": "2024-01-15T10:30:00"
  }
}
```

GET /api/user/accounts

List all accounts for the authenticated user.

Headers: `Authorization: Bearer <token>`

Response (200 OK):

```
{
  "success": true,
  "message": "Accounts retrieved successfully",
  "data": [
    {
      "accountId": "uuid",
      "accountNumber": "JB123456789012345678",
      "accountType": "SAVINGS",
      "balance": 5000.00,
      "status": "Active"
    }
  ]
}
```

GET /api/user/accounts/{accountId}

Get details of a specific account.

Headers: `Authorization: Bearer <token>`

Response (200 OK):

```
{
  "success": true,
  "message": "Account details retrieved successfully",
  "data": {
```

```
    "accountId": "uuid",
    "accountNumber": "JB123456789012345678",
    "accountType": "SAVINGS",
    "balance": 5000.00,
    "status": "Active",
    "holder": {
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com"
    },
    "createdAt": "2024-01-15T10:30:00"
  }
}
```

PUT /api/user/accounts/{accountId}

Update account holder details.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "firstName": "John",
  "lastName": "Smith",
  "address": "456 New Street, City, State 12345",
  "phone": "+9876543210"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Account updated successfully",
  "data": { ... }
}
```

PUT /api/user/accounts/{accountId}/close

Close an account (soft delete).

Headers: `Authorization: Bearer <token>`

Response (200 OK):

```
{
  "success": true,
  "message": "Account closed successfully",
  "data": { ... }
}
```

DELETE /api/user/accounts/{accountId}

Permanently delete a closed account.

Headers: `Authorization: Bearer <token>`

Note: Account must be closed and have zero balance.

Response (200 OK):

```
{
  "success": true,
  "message": "Account deleted successfully",
  "data": null
}
```

3.3 User Transaction Endpoints

POST /api/user/accounts/{accountId}/deposit

Deposit money into account.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "amount": 1000.00,
  "description": "Salary deposit"
}
```

Validation:

- Amount must be greater than 0
- Account must be "Active"

Response (200 OK):

```
{
  "success": true,
  "message": "Deposit successful",
  "data": {
    "transactionId": 12345,
    "accountNumber": "JB123456789012345678",
    "transactionType": "CREDIT",
    "amount": 1000.00,
    "newBalance": 6000.00,
    "description": "Salary deposit",
    "timestamp": "2024-01-15T10:30:00"
  }
}
```

POST /api/user/accounts/{accountId}/withdraw

Withdraw money from account.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "amount": 500.00,
  "description": "ATM withdrawal"
}
```

Validation:

- Amount must be greater than 0
- Account must be "Active"
- Sufficient balance required

Response (200 OK):

```
{
  "success": true,
  "message": "Withdrawal successful",
  "data": {
    "transactionId": 12346,
    "accountNumber": "JB123456789012345678",
    "transactionType": "DEBIT",
    "amount": 500.00,
    "newBalance": 5500.00,
    "description": "ATM withdrawal",
    "timestamp": "2024-01-15T10:35:00"
  }
}
```

POST /api/user/accounts/{accountId}/transfer

Transfer money to another account.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "toAccountNumber": "JB987654321098765432",
  "amount": 200.00,
  "description": "Payment to vendor"
}
```

Validation:

- Amount must be greater than 0

- Source account must be "Active"
- Sufficient balance required
- Cannot transfer to self

Response (200 OK):

```
{
  "success": true,
  "message": "Transfer successful",
  "data": {
    "transactionId": 12347,
    "fromAccountNumber": "JB123456789012345678",
    "toAccountNumber": "JB987654321098765432",
    "amount": 200.00,
    "newBalance": 5300.00,
    "description": "Payment to vendor",
    "timestamp": "2024-01-15T10:40:00"
  }
}
```

GET /api/user/accounts/{accountId}/transactions

Get transaction history for an account.

Headers: `Authorization: Bearer <token>`

Response (200 OK):

```
{
  "success": true,
  "message": "Transactions retrieved successfully",
  "data": [
    {
      "transactionId": 12347,
      "transactionType": "DEBIT",
      "amount": 200.00,
      "description": "Payment to vendor",
      "relatedAccountNumber": "JB987654321098765432",
      "status": "Completed",
      "timestamp": "2024-01-15T10:40:00"
    },
    {
      "transactionId": 12346,
      "transactionType": "DEBIT",
      "amount": 500.00,
      "description": "ATM withdrawal",
      "status": "Completed",
      "timestamp": "2024-01-15T10:35:00"
    }
  ]
}
```

GET /api/user/accounts/{accountId}/transactions/paginated

Get paginated transaction history.

Headers: `Authorization: Bearer <token>`

Query Parameters:

- `page` (default: 0)
- `size` (default: 10)

Example: `/api/user/accounts/{id}/transactions/paginated?page=0&size=10`

Response (200 OK):

```
{
  "success": true,
  "message": "Transactions retrieved successfully",
  "data": {
    "content": [ ... ],
    "page": 0,
    "size": 10,
    "totalElements": 50,
    "totalPages": 5
  }
}
```

3.4 User Loan Endpoints

POST /api/user/applyloan

Apply for a loan.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "loanType": "HOME",
  "principalAmount": 500000.00,
  "termMonths": 240,
  "reason": "Home purchase"
}
```

Loan Types: `HOME` , `PERSONAL` , `AUTO` , `EDUCATION`

Validation:

- principalAmount > 0
- termMonths > 0

Response (201 Created):

```
{
  "success": true,
  "message": "Loan application submitted successfully",
  "data": {
    "loanId": "uuid",
    "loanType": "HOME",
    "principalAmount": 500000.00,
    "interestRate": 8.5,
    "termMonths": 240,
    "monthlyPayment": 4349.00,
    "status": "Pending",
    "applicationDate": "2024-01-15"
  }
}
```

GET /api/user/viewloan

View all loans for the authenticated user.

Headers: `Authorization: Bearer <token>`

Response (200 OK):


```
{
  "success": true,
  "message": "Loans retrieved successfully",
  "data": [
    {
      "loanId": "uuid",
      "loanType": "HOME",
      "principalAmount": 500000.00,
      "currentBalance": 500000.00,
      "interestRate": 8.5,
      "termMonths": 240,
      "status": "Approved",
      "startDate": "2024-01-15",
      "endDate": "2044-01-15"
    }
  ]
}
```

3.5 User Card Endpoints

POST /api/user/applycard

Apply for a debit or credit card.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "accountId": "550e8400-e29b-41d4-a716-446655440000",
  "cardType": "DEBIT"
}
```

Card Types: `DEBIT` , `CREDIT`

Important: CVV is returned ONLY at card creation. User must save it!

Validation:

- Account must be "Active"
- Maximum 3 cards per account

Response (201 Created):

```
{
  "success": true,
  "message": "Card application submitted successfully",
  "data": {
    "cardId": "uuid",
    "accountId": "550e8400-e29b-41d4-a716-446655440000",
    "accountNumber": "JB123456789012345678",
    "cardNumber": "1234567890123456",
    "cvv": "123",
    "cardType": "DEBIT",
    "expirationDate": "2027-01-15",
    "dailyLimit": 500.00,
    "status": "Pending",
    "issueDate": "2024-01-15",
    "holderName": "John Doe"
  }
}
```

GET /api/user/viewcard

View all cards for the authenticated user.

Headers: `Authorization: Bearer <token>`

Note: CVV is not returned in this response.

Response (200 OK):

```
{
  "success": true,
  "message": "Cards retrieved successfully",
  "data": [
    {
      "cardId": "uuid",
      "accountId": "550e8400-e29b-41d4-a716-446655440000",
      "accountNumber": "JB123456789012345678",
```

```
      "cardNumber": "1234567890123456",
      "cvv": null,
      "cardType": "DEBIT",
      "expirationDate": "2027-01-15",
      "dailyLimit": 500.00,
      "status": "Active",
      "issueDate": "2024-01-15",
      "holderName": "John Doe"
    }
  ]
}
```

3.6 User Chequebook Endpoints

POST /api/user/applychequebook

Request a cheque book.

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "accountId": "550e8400-e29b-41d4-a716-446655440000",
  "numberOfLeaves": 50,
  "deliveryAddress": "123 Main Street, City, State 12345"
}
```

Validation:

- numberOfLeaves: 1-50 (default: 50)
- Account must be "Active"

Response (201 Created):

```
{
  "success": true,
  "message": "ChequeBook request submitted successfully",
  "data": {
    "requestId": 123,
    "accountId": "550e8400-e29b-41d4-a716-446655440000",
    "accountNumber": "JB123456789012345678",
    "requestDate": "2024-01-15T10:45:00",
    "numberOfLeaves": 50,
    "deliveryAddress": "123 Main Street, City, State 12345",
    "status": "Pending",
    "holderName": "John Doe"
  }
}
```

GET /api/user/viewchequebook

View all chequebook requests for the authenticated user.

Headers: `Authorization: Bearer <token>`

Response (200 OK):

```
{
  "success": true,
  "message": "ChequeBook requests retrieved successfully",
  "data": [
    {
      "requestId": 123,
      "accountId": "uuid",
      "accountNumber": "JB123456789012345678",
      "requestDate": "2024-01-15T10:45:00",
      "numberOfLeaves": 50,
      "deliveryAddress": "123 Main Street, City, State 12345",
      "status": "Pending",
      "holderName": "John Doe"
    }
  ]
}
```

3.7 Admin Account Endpoints

POST /api/admin/accounts/open

Open account for existing user.

Headers: `Authorization: Bearer <admin-token>`

Request:

```
{
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "accountType": "SAVINGS"
}
```

Response (201 Created):

```
{
  "success": true,
  "message": "Account opened successfully",
  "data": { ... }
}
```

GET /api/admin/accounts

List all accounts.

Headers: `Authorization: Bearer <admin-token>`

Query Parameters (optional):

- `holderId` : Filter by account holder

Response (200 OK):

```
{
  "success": true,
  "message": "All accounts retrieved successfully",
  "data": [ ... ]
}
```

GET /api/admin/accounts/{accountId}

Get any account details.

Headers: `Authorization: Bearer <admin-token>`

PUT /api/admin/accounts/{accountId}

Update any account.

Headers: `Authorization: Bearer <admin-token>`

PUT /api/admin/accounts/{accountId}/close

Close any account.

Headers: `Authorization: Bearer <admin-token>`

DELETE /api/admin/accounts/{accountId}

Delete any closed account.

Headers: `Authorization: Bearer <admin-token>`

POST /api/admin/accounts/{accountId}/deposit

Admin deposit to any account.

Headers: `Authorization: Bearer <admin-token>`

POST /api/admin/accounts/{accountId}/withdraw

Admin withdrawal from any account.

Headers: `Authorization: Bearer <admin-token>`

POST /api/admin/accounts/{accountId}/transfer

Admin transfer from any account.

Headers: `Authorization: Bearer <admin-token>`

3.8 Admin User Management Endpoints

GET /api/admin/users

List all users.

Headers: `Authorization: Bearer <admin-token>`

Response (200 OK):


```
{
  "success": true,
  "message": "Users retrieved successfully",
  "data": [
    {
      "userId": "uuid",
      "username": "johndoe",
      "email": "john@example.com",
      "role": "ROLE_USER",
      "isActive": true,
      "createdAt": "2024-01-10T08:00:00"
    }
  ]
}
```

GET /api/admin/users/{userId}

Get specific user details.

Headers: `Authorization: Bearer <admin-token>`

PUT /api/admin/users/{userId}/activate

Activate a user.

Headers: `Authorization: Bearer <admin-token>`

PUT /api/admin/users/{userId}/deactivate

Deactivate a user.

Headers: `Authorization: Bearer <admin-token>`

PUT /api/admin/users/{userId}/role

Change user role.

Headers: `Authorization: Bearer <admin-token>`

Request:

```
{
  "role": "ADMIN"
}
```

Roles: `USER` , `ADMIN`

3.9 Admin Transaction Endpoints

GET /api/admin/transactions

View all transactions (admin).

Headers: `Authorization: Bearer <admin-token>`

Query Parameters (optional):

- `accountNumber` : Filter by account

3.10 Admin Loan Endpoints

GET /api/admin/loans

View all loan applications.

Headers: `Authorization: Bearer <admin-token>`

Response (200 OK):

```
{
  "success": true,
  "message": "Loans retrieved successfully",
  "data": [
    {
      "loanId": "uuid",
      "loanType": "HOME",
      "principalAmount": 500000.00,
      "holderName": "John Doe",
      "status": "Pending",
      "applicationDate": "2024-01-15"
    }
  ]
}
```

POST /api/admin/loans/{loanId}/approve

Approve a loan application.

Headers: `Authorization: Bearer <admin-token>`

Note: Approving a loan disburses funds to the user's account.

POST /api/admin/loans/{loanId}/reject

Reject a loan application.

Headers: `Authorization: Bearer <admin-token>`

Request:

```
{
  "reason": "Insufficient income documentation"
}
```

3.11 Admin Card Endpoints

GET /api/admin/cards

View all card applications.

Headers: `Authorization: Bearer <admin-token>`

Response (200 OK):

```
{
  "success": true,
  "message": "All cards retrieved successfully",
  "data": [
    {
      "cardId": "uuid",
      "accountId": "uuid",
      "accountNumber": "JB123456789012345678",
      "cardNumber": "1234567890123456",
      "cardType": "DEBIT",
      "status": "Pending",
      "holderName": "John Doe",
      "issueDate": "2024-01-15"
    }
  ]
}
```

POST /api/admin/cards/{cardId}/approve

Approve a card application.

Headers: `Authorization: Bearer <admin-token>`

POST /api/admin/cards/{cardId}/reject

Reject a card application.

Headers: `Authorization: Bearer <admin-token>`

Query Parameters:

- `reason` (optional, default: "Rejected by admin")

3.12 Admin Chequebook Endpoints

GET /api/admin/chequebooks

View all chequebook requests.

Headers: `Authorization: Bearer <admin-token>`

POST /api/admin/chequebooks/{requestId}/approve

Approve a chequebook request.

Headers: `Authorization: Bearer <admin-token>`

POST /api/admin/chequebooks/{requestId}/reject

Reject a chequebook request.

Headers: `Authorization: Bearer <admin-token>`

Query Parameters:

- `reason` (optional, default: "Rejected by admin")

---

4. Request/Response Schemas

4.1 Authentication DTOs

LoginRequest

```
interface LoginRequest {
  username: string;  // Required
  password: string;  // Required
}
```

RegisterRequest

```
interface RegisterRequest {
  username: string;      // Required, unique
  password: string;      // Required, min 8 chars
  email: string;         // Required, valid email, unique
  firstName: string;     // Required
  lastName: string;      // Required
}
```

JwtResponse

```typescript
interface JwtResponse {
  token: string;          // JWT token
  userId: string;         // UUID
  username: string;
  email: string;
  role: string;           // "ROLE_USER" or "ROLE_ADMIN"
}
```

4.2 Account DTOs

OpenAccountRequest

```typescript
interface OpenAccountRequest {
  firstName?: string;        // Optional for existing holders
  lastName?: string;          // Optional for existing holders
  dateOfBirth?: string;      // ISO date format "YYYY-MM-DD"
  address?: string;           // Optional
  phone?: string;             // Optional
  citizenshipId?: string;     // Optional
  accountType: string;        // Required: SAVINGS, CHECKING, FIXED_DEPOSIT
}
```

AccountResponse

```typescript
interface AccountResponse {
  accountId: string;          // UUID
  accountNumber: string;      // e.g., "JB123456789012345678"
  accountType: string;
  balance: number;
  status: string;             // Active, Closed
  createdAt?: string;         // ISO datetime
}
```

AccountDetailsResponse

```typescript
interface AccountDetailsResponse {
  accountId: string;
  accountNumber: string;
  accountType: string;
  balance: number;
  status: string;
  holder: {
    firstName: string;
    lastName: string;
    email: string;
  };
```

```
  createdAt: string;
}
```

4.3 Transaction DTOs

DepositRequest

```
interface DepositRequest {
  amount: number;              // Required, > 0
  description?: string;         // Optional
}
```

WithdrawRequest

```
interface WithdrawRequest {
  amount: number;              // Required, > 0
  description?: string;        // Optional
}
```

TransferRequest

```
interface TransferRequest {
  toAccountNumber: string;     // Required
  amount: number;              // Required, > 0
  description?: string;       // Optional
}
```

TransactionResponse

```
interface TransactionResponse {
  transactionId: number;
  accountNumber: string;
  transactionType: string;     // CREDIT, DEBIT
  amount: number;
  newBalance?: number;          // After transaction
  description?: string;
  relatedAccountNumber?: string;  // For transfers
  status: string;               // Completed, Failed
  timestamp: string;            // ISO datetime
}
```

4.4 Loan DTOs

ApplyLoanRequest

```
interface ApplyLoanRequest {
  loanType: string;            // Required: HOME, PERSONAL, AUTO, EDUCATION
  principalAmount: number;     // Required, > 0
  termMonths: number;          // Required, > 0
  reason?: string;             // Optional
}
```

LoanResponse

```
interface LoanResponse {
  loanId: string;              // UUID
  loanType: string;
  principalAmount: number;
  currentBalance: number;
  interestRate: number;        // Annual percentage
  termMonths: number;
  monthlyPayment: number;
  status: string;              // Pending, Approved, Rejected
  applicationDate?: string;
  startDate?: string;
  endDate?: string;
  holderName?: string;
}
```

4.5 Card DTOs

ApplyCardRequest

```
interface ApplyCardRequest {
  accountId: string;           // UUID, Required
  cardType: string;           // Required: CREDIT, DEBIT
}
```

CardResponse

```
interface CardResponse {
  cardId: string;             // UUID
  accountId: string;
  accountNumber: string;
  cardNumber: string;          // 16 digits
  cvv: string | null;          // Only returned at creation!
  cardType: string;
  expirationDate: string;     // "YYYY-MM-DD"
  dailyLimit: number;
  status: string;              // Pending, Active, Rejected
  issueDate: string;          // ISO date
```

```
  holderName: string;
}
```

4.6 Chequebook DTOs

ApplyChequeBookRequest

```
interface ApplyChequeBookRequest {
  accountId: string;           // UUID, Required
  numberOfLeaves?: number;     // 1-50, default 50
  deliveryAddress?: string;    // Optional
}
```

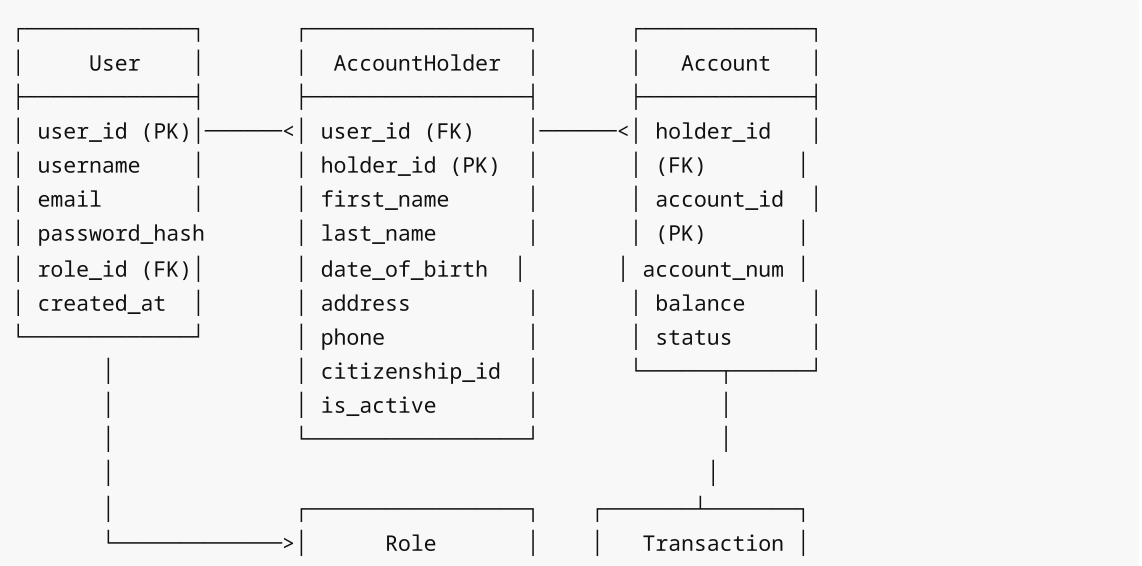
ChequeBookResponse

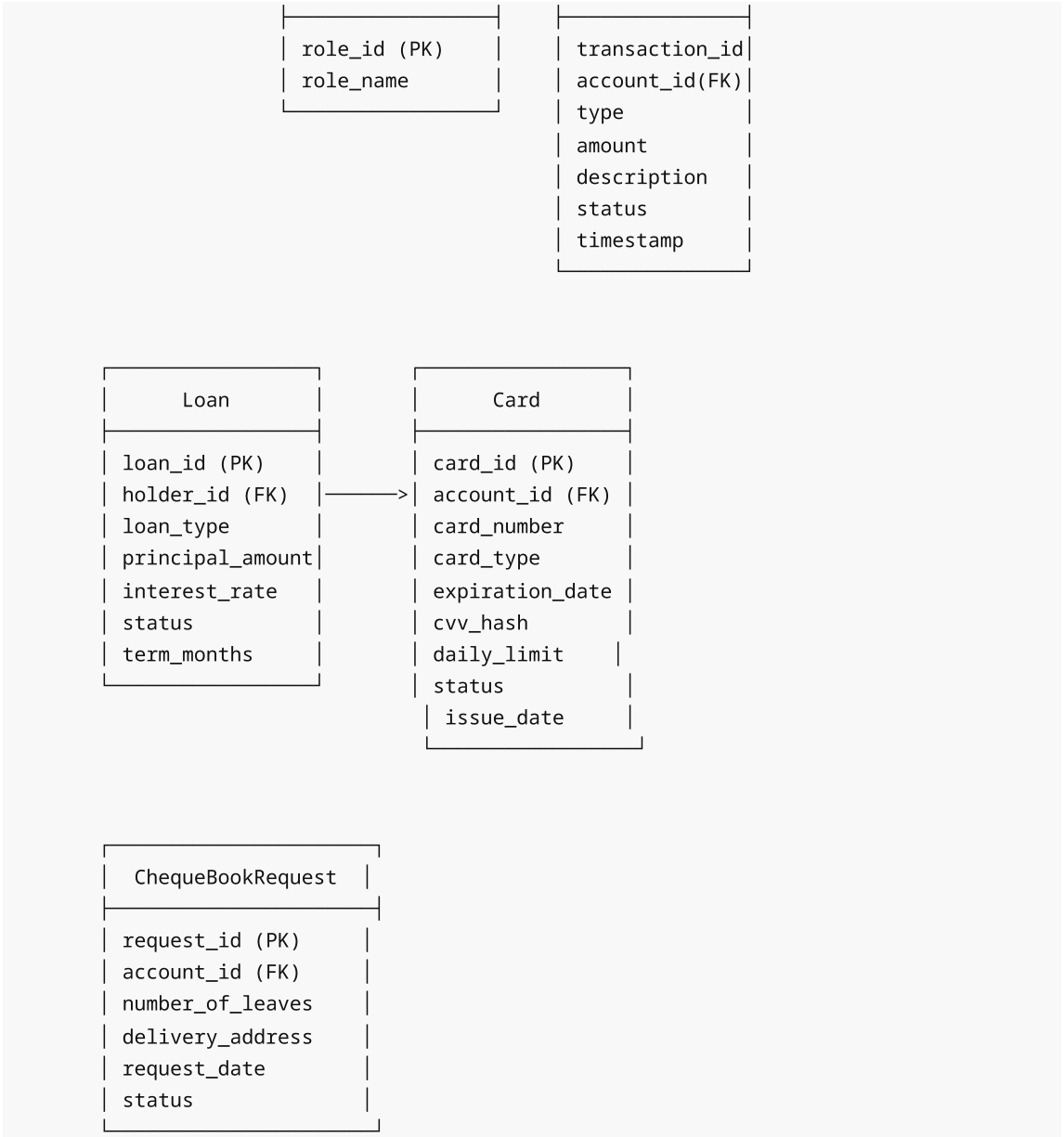
```
interface ChequeBookResponse {
  requestId: number;
  accountId: string;
  accountNumber: string;
  requestDate: string;         // ISO datetime
  numberOfLeaves: number;
  deliveryAddress: string;
  status: string;              // Pending, Approved, Rejected
  holderName: string;
}
```

---

5. Database Schema

5.1 Entity Relationship Diagram





5.2 Table Definitions

Users Table

```
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role_id INTEGER NOT NULL REFERENCES role(role_id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Role Table

```
CREATE TABLE role (
    role_id SERIAL PRIMARY KEY,
    role_name VARCHAR(20) UNIQUE NOT NULL
);
-- Values: 'ROLE_USER', 'ROLE_ADMIN'
```

AccountHolder Table

```
CREATE TABLE account_holder (
    holder_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID UNIQUE NOT NULL REFERENCES users(user_id),
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    date_of_birth DATE,
    address TEXT,
    phone VARCHAR(20),
    citizenship_id VARCHAR(50) UNIQUE,
    is_active BOOLEAN DEFAULT TRUE
);
```

Account Table

```
CREATE TABLE account (
    account_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_number VARCHAR(20) UNIQUE NOT NULL,
    holder_id UUID NOT NULL REFERENCES account_holder(holder_id),
    account_type VARCHAR(50) NOT NULL,
    balance DECIMAL(15,2) DEFAULT 0.00,
    status VARCHAR(20) DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Transaction Table

```
CREATE TABLE transaction (
    transaction_id BIGSERIAL PRIMARY KEY,
    account_id UUID NOT NULL REFERENCES account(account_id),
    transaction_type VARCHAR(10) NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    description TEXT,
    related_account_number VARCHAR(20),
    status VARCHAR(20) DEFAULT 'Completed',
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Loan Table

```
CREATE TABLE loan (
    loan_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    holder_id UUID NOT NULL REFERENCES account_holder(holder_id),
    loan_type VARCHAR(20) NOT NULL,
    principal_amount DECIMAL(15,2) NOT NULL,
    current_balance DECIMAL(15,2) NOT NULL,
    interest_rate DECIMAL(5,4),
    term_months INTEGER NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending',
    start_date DATE,
    end_date DATE
);
```

Card Table

```
CREATE TABLE card (
    card_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id UUID NOT NULL REFERENCES account(account_id),
    card_number VARCHAR(16) UNIQUE NOT NULL,
    card_type VARCHAR(20) NOT NULL,
    expiration_date DATE NOT NULL,
    cvv_hash VARCHAR(100),
    daily_limit DECIMAL(10,2) DEFAULT 500.00,
    status VARCHAR(20) DEFAULT 'Pending',
    issue_date DATE DEFAULT CURRENT_DATE
);
```

ChequeBookRequest Table

```
CREATE TABLE chequebook_request (
    request_id SERIAL PRIMARY KEY,
    account_id UUID NOT NULL REFERENCES account(account_id),
    number_of_leaves INTEGER DEFAULT 50,
    delivery_address TEXT,
    request_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Pending'
);
```

---

6. Error Handling

6.1 HTTP Status Codes

Code	Meaning	When Used

200	OK	Successful GET, PUT operations
201	Created	Successful POST (creation)
400	Bad Request	Validation errors, invalid input
401	Unauthorized	Missing or invalid JWT token
403	Forbidden	Insufficient permissions
404	Not Found	Resource doesn't exist
500	Internal Server Error	Server-side errors

6.2 Common Error Messages

Message	Meaning	Solution
"Invalid credentials"	Wrong username/password	Check login credentials
"Access denied"	Not authorized for this resource	Verify account ownership
"Account not found"	Account doesn't exist	Check account ID
"Insufficient balance"	Not enough funds	Deposit funds first
"Account is not active"	Account is closed/suspended	Contact support
"Maximum card limit reached"	3 cards already on account	Cannot add more cards
"Only pending requests can be approved"	Wrong status	Check request status

6.3 Validation Errors

When request validation fails, the response includes field-specific errors:

```
{
  "success": false,
  "message": "Validation failed",
  "data": {
    "username": "Username is required",
    "email": "Invalid email format",
    "amount": "Amount must be greater than 0"
  }
}
```

---

7. React Integration Guide

7.1 Project Setup

Install Dependencies

```
npm install axios
npm install react-router-dom
npm install @tanstack/react-query  # Optional but recommended
```

Environment Configuration

Create `.env` file:

```
REACT_APP_API_BASE_URL=http://localhost:8080
```

7.2 API Service Layer

api/axios.ts

```
import axios from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_BASE_URL || 'http://localhost:8080';

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor to add auth token
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor for error handling
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Token expired or invalid
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
```

```
    }
    return Promise.reject(error);
  }
);

export default api;
```

7.3 Authentication Context

context/AuthContext.tsx

```
import React, { createContext, useContext, useState, useEffect } from 'react';
import api from '../services/api';

interface User {
  userId: string;
  username: string;
  email: string;
  role: string;
}

interface AuthContextType {
  user: User | null;
  token: string | null;
  login: (username: string, password: string) => Promise<void>;
  register: (data: RegisterData) => Promise<void>;
  logout: () => void;
  isAuthenticated: boolean;
  isAdmin: boolean;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children })
=> {
  const [user, setUser] = useState<User | null>(null);
  const [token, setToken] = useState<string | null>(localStorage.getItem('token'));

  useEffect(() => {
    const storedUser = localStorage.getItem('user');
    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  const login = async (username: string, password: string) => {
    const response = await api.post('/api/auth/login', { username, password });
    const { token, userId, username: name, email, role } = response.data.data;

    localStorage.setItem('token', token);
```

```
    localStorage.setItem('user', JSON.stringify({ userId, username: name, email,
role }));

    setToken(token);
    setUser({ userId, username: name, email, role });
  };

  const register = async (data: RegisterData) => {
    await api.post('/api/auth/register', data);
  };

  const logout = () => {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    setToken(null);
    setUser(null);
  };

  return (
    <AuthContext.Provider
      value={{
        user,
        token,
        login,
        register,
        logout,
        isAuthenticated: !!token,
        isAdmin: user?.role === 'ROLE_ADMIN',
      }}
    >
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within AuthProvider');
  return context;
};
```

7.4 Authentication Services

services/authService.ts

```
import api from './axios';

interface LoginCredentials {
  username: string;
  password: string;
```

```
}

interface RegisterData {
  username: string;
  password: string;
  email: string;
  firstName: string;
  lastName: string;
}

interface AuthResponse {
  token: string;
  userId: string;
  username: string;
  email: string;
  role: string;
}

export const authService = {
  login: async (credentials: LoginCredentials): Promise<AuthResponse> => {
    const response = await api.post('/api/auth/login', credentials);
    return response.data.data;
  },

  register: async (data: RegisterData): Promise<void> => {
    await api.post('/api/auth/register', data);
  },
};
```

7.5 Account Services

services/accountService.ts

```
import api from './axios';

export interface OpenAccountData {
  firstName?: string;
  lastName?: string;
  dateOfBirth?: string;
  address?: string;
  phone?: string;
  citizenshipId?: string;
  accountType: 'SAVINGS' | 'CHECKING' | 'FIXED_DEPOSIT';
}

export interface Account {
  accountId: string;
  accountNumber: string;
  accountType: string;
  balance: number;
```

```
  status: string;
}

export const accountService = {
  openAccount: async (data: OpenAccountData): Promise<Account> => {
    const response = await api.post('/api/user/openaccount', data);
    return response.data.data;
  },

  getAccounts: async (): Promise<Account[]> => {
    const response = await api.get('/api/user/accounts');
    return response.data.data;
  },

  getAccount: async (accountId: string): Promise<Account> => {
    const response = await api.get(`/api/user/accounts/${accountId}`);
    return response.data.data;
  },

  closeAccount: async (accountId: string): Promise<void> => {
    await api.put(`/api/user/accounts/${accountId}/close`);
  },

  deleteAccount: async (accountId: string): Promise<void> => {
    await api.delete(`/api/user/accounts/${accountId}`);
  },
};
```

7.6 Transaction Services

services/transactionService.ts

```
import api from './axios';

export interface DepositData {
  amount: number;
  description?: string;
}

export interface WithdrawData {
  amount: number;
  description?: string;
}

export interface TransferData {
  toAccountNumber: string;
  amount: number;
  description?: string;
}
```

```ts
export interface Transaction {
  transactionId: number;
  transactionType: string;
  amount: number;
  description?: string;
  relatedAccountNumber?: string;
  status: string;
  timestamp: string;
}

export const transactionService = {
  deposit: async (accountId: string, data: DepositData): Promise<Transaction> => {
    const response = await api.post(`/api/user/accounts/${accountId}/deposit`,
data);
    return response.data.data;
  },

  withdraw: async (accountId: string, data: WithdrawData): Promise<Transaction> => {
    const response = await api.post(`/api/user/accounts/${accountId}/withdraw`,
data);
    return response.data.data;
  },

  transfer: async (accountId: string, data: TransferData): Promise<Transaction> => {
    const response = await api.post(`/api/user/accounts/${accountId}/transfer`,
data);
    return response.data.data;
  },

  getTransactions: async (accountId: string): Promise<Transaction[]> => {
    const response = await api.get(`/api/user/accounts/${accountId}/transactions`);
    return response.data.data;
  },

  getPaginatedTransactions: async (
    accountId: string,
    page: number = 0,
    size: number = 10
  ): Promise<{ content: Transaction[]; totalElements: number }> => {
    const response = await api.get(
      `/api/user/accounts/${accountId}/transactions/paginated?
page=${page}&size=${size}`
    );
    return response.data.data;
  },
};
```

7.7 Loan Services

services/loanService.ts

```ts
import api from './axios';

export interface ApplyLoanData {
  loanType: 'HOME' | 'PERSONAL' | 'AUTO' | 'EDUCATION';
  principalAmount: number;
  termMonths: number;
  reason?: string;
}

export interface Loan {
  loanId: string;
  loanType: string;
  principalAmount: number;
  currentBalance: number;
  interestRate: number;
  termMonths: number;
  monthlyPayment: number;
  status: string;
  applicationDate?: string;
  startDate?: string;
  endDate?: string;
}

export const loanService = {
  applyLoan: async (data: ApplyLoanData): Promise<Loan> => {
    const response = await api.post('/api/user/applyloan', data);
    return response.data.data;
  },

  getLoans: async (): Promise<Loan[]> => {
    const response = await api.get('/api/user/viewloan');
    return response.data.data;
  },
};
```

7.8 Card Services

services/cardService.ts

```ts
import api from './axios';

export interface ApplyCardData {
  accountId: string;
  cardType: 'CREDIT' | 'DEBIT';
}

export interface Card {
  cardId: string;
  accountId: string;
```

```
  accountNumber: string;
  cardNumber: string;
  cvv: string | null;  // Only available at creation!
  cardType: string;
  expirationDate: string;
  dailyLimit: number;
  status: string;
  issueDate: string;
  holderName: string;
}

export const cardService = {
  applyCard: async (data: ApplyCardData): Promise<Card> => {
    const response = await api.post('/api/user/applycard', data);
    return response.data.data;
  },

  getCards: async (): Promise<Card[]> => {
    const response = await api.get('/api/user/viewcard');
    return response.data.data;
  },
};
```

7.9 Protected Route Component

components/ProtectedRoute.tsx

```
import React from 'react';
import { Navigate, Route, RouteProps } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

interface ProtectedRouteProps extends RouteProps {
  requiredRole?: 'USER' | 'ADMIN';
}

export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({
  children,
  requiredRole,
}) => {
  const { isAuthenticated, isAdmin } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (requiredRole === 'ADMIN' && !isAdmin) {
    return <Navigate to="/dashboard" replace />;
  }

  return <>{children}</>;
```

```
};

// Usage
<Route
  path="/admin"
  element={
    <ProtectedRoute requiredRole="ADMIN">
      <AdminDashboard />
    </ProtectedRoute>
  }
/>
```

7.10 Example Components

Login Component

```
import React, { useState } from 'react';
import { useAuth } from '../context/AuthContext';

export const Login: React.FC = () => {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const { login } = useAuth();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      setError('');
      await login(username, password);
      // Redirect will be handled by AuthContext or router
    } catch (err: any) {
      setError(err.response?.data?.message || 'Login failed');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
        placeholder="Username"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
```

```
      {error && <p>{error}</p>}
      <button type="submit">Login</button>
    </form>
  );
};
```

Deposit Component

```
import React, { useState } from 'react';
import { accountService, transactionService } from '../services';

export const Deposit: React.FC<{ accountId: string }> = ({ accountId }) => {
  const [amount, setAmount] = useState('');
  const [description, setDescription] = useState('');
  const [message, setMessage] = useState('');

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      const transaction = await transactionService.deposit(accountId, {
        amount: parseFloat(amount),
        description,
      });
      setMessage(`Deposit successful! New balance: $${transaction.newBalance}`);
      setAmount('');
      setDescription('');
    } catch (err: any) {
      setMessage(err.response?.data?.message || 'Deposit failed');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        placeholder="Amount"
        required
      />
      <input
        type="text"
        value={description}
        onChange={(e) => setDescription(e.target.value)}
        placeholder="Description (optional)"
      />
      <button type="submit">Deposit</button>
      {message && <p>{message}</p>}
    </form>
```

```
  );
};
```

---

8. Testing Guide

8.1 Authentication Tests

Register a new user

```
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "testuser",
    "password": "TestPass123!",
    "email": "testuser@example.com",
    "firstName": "John",
    "lastName": "Doe"
  }'
```

Login

```
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "username": "testuser",
    "password": "TestPass123!"
  }'
```

8.2 Account Tests

Open account

```
curl -X POST http://localhost:8080/api/user/openaccount \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "firstName": "John",
    "lastName": "Doe",
    "accountType": "SAVINGS"
  }'
```

List accounts

```
curl -X GET http://localhost:8080/api/user/accounts \
```

```
  -H "Authorization: Bearer $TOKEN"
```

Get account details

```
curl -X GET http://localhost:8080/api/user/accounts/{accountId} \
  -H "Authorization: Bearer $TOKEN"
```

8.3 Transaction Tests

Deposit

```
curl -X POST http://localhost:8080/api/user/accounts/{accountId}/deposit \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "amount": 1000.00,
    "description": "Salary deposit"
  }'
```

Withdraw

```
curl -X POST http://localhost:8080/api/user/accounts/{accountId}/withdraw \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "amount": 500.00,
    "description": "ATM withdrawal"
  }'
```

Transfer

```
curl -X POST http://localhost:8080/api/user/accounts/{accountId}/transfer \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "toAccountNumber": "JB9876543210987654321",
    "amount": 200.00,
    "description": "Payment"
  }'
```

View transactions

```
curl -X GET http://localhost:8080/api/user/accounts/{accountId}/transactions \
  -H "Authorization: Bearer $TOKEN"
```

View paginated transactions

```
curl -X GET
"http://localhost:8080/api/user/accounts/{accountId}/transactions/paginated?
page=0&size=10" \
  -H "Authorization: Bearer $TOKEN"
```

8.4 Loan Tests

Apply for loan

```
curl -X POST http://localhost:8080/api/user/applyloan \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "loanType": "HOME",
    "principalAmount": 500000.00,
    "termMonths": 240,
    "reason": "Home purchase"
  }'
```

View loans

```
curl -X GET http://localhost:8080/api/user/viewloan \
  -H "Authorization: Bearer $TOKEN"
```

8.5 Card Tests

Apply for card

```
curl -X POST http://localhost:8080/api/user/applycard \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "accountId": "550e8400-e29b-41d4-a716-446655440000",
    "cardType": "DEBIT"
  }'
```

View cards

```
curl -X GET http://localhost:8080/api/user/viewcard \
  -H "Authorization: Bearer $TOKEN"
```

8.6 Chequebook Tests

Apply for chequebook

```
curl -X POST http://localhost:8080/api/user/applychequebook \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "accountId": "550e8400-e29b-41d4-a716-446655440000",
    "numberOfLeaves": 50,
    "deliveryAddress": "123 Main Street, City, State 12345"
  }'
```

View chequebook requests

```
curl -X GET http://localhost:8080/api/user/viewchequebook \
  -H "Authorization: Bearer $TOKEN"
```

8.7 Admin Tests

View all accounts

```
curl -X GET http://localhost:8080/api/admin/accounts \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

View all loans

```
curl -X GET http://localhost:8080/api/admin/loans \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

Approve loan

```
curl -X POST http://localhost:8080/api/admin/loans/{loanId}/approve \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

Reject loan

```
curl -X POST http://localhost:8080/api/admin/loans/{loanId}/reject \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{
    "reason": "Insufficient documentation"
  }'
```

View all cards

```
curl -X GET http://localhost:8080/api/admin/cards \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

Approve card

```
curl -X POST http://localhost:8080/api/admin/cards/{cardId}/approve \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

Reject card

```
curl -X POST "http://localhost:8080/api/admin/cards/{cardId}/reject?
reason=Account%20not%20eligible" \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

View all chequebooks

```
curl -X GET http://localhost:8080/api/admin/chequebooks \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

Approve chequebook

```
curl -X POST http://localhost:8080/api/admin/chequebooks/{requestId}/approve \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

Manage users

```
# View all users
curl -X GET http://localhost:8080/api/admin/users \
  -H "Authorization: Bearer $ADMIN_TOKEN"

# Deactivate user
curl -X PUT http://localhost:8080/api/admin/users/{userId}/deactivate \
  -H "Authorization: Bearer $ADMIN_TOKEN"

# Change role
curl -X PUT http://localhost:8080/api/admin/users/{userId}/role \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"role": "ADMIN"}'
```

8.8 Security Tests

Access without token (should return 401)

```
curl -X GET http://localhost:8080/api/user/accounts
```

User accessing admin endpoint (should return 403)

```
curl -X GET http://localhost:8080/api/admin/accounts \
  -H "Authorization: Bearer $USER_TOKEN"
```

Invalid credentials

```
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "username": "wronguser",
    "password": "wrongpass"
  }'
```

Invalid account type

```
curl -X POST http://localhost:8080/api/user/openaccount \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "firstName": "John",
    "lastName": "Doe",
    "accountType": "INVALID"
  }'
```

Appendix A: Quick Reference

A.1 Endpoint Summary

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login
POST	/api/user/openaccount	USER	Open account
GET	/api/user/accounts	USER	List accounts
GET	/api/user/accounts/{id}	USER	Get account
PUT	/api/user/accounts/{id}	USER	Update account

PUT	/api/user/accounts/{id}/close	USER	Close account
DELETE	/api/user/accounts/{id}	USER	Delete account
POST	/api/user/accounts/{id}/deposit	USER	Deposit
POST	/api/user/accounts/{id}/withdraw	USER	Withdraw
POST	/api/user/accounts/{id}/transfer	USER	Transfer
GET	/api/user/accounts/{id}/transactions	USER	View transactions
POST	/api/user/applyloan	USER	Apply loan
GET	/api/user/viewloan	USER	View loans
POST	/api/user/applycard	USER	Apply card
GET	/api/user/viewcard	USER	View cards
POST	/api/user/applychequebook	USER	Apply chequebook
GET	/api/user/viewchequebook	USER	View chequebooks
GET	/api/admin/accounts	ADMIN	List all accounts
GET	/api/admin/accounts/{id}	ADMIN	Get account
POST	/api/admin/accounts/open	ADMIN	Open account
PUT	/api/admin/accounts/{id}	ADMIN	Update account
PUT	/api/admin/accounts/{id}/close	ADMIN	Close account
DELETE	/api/admin/accounts/{id}	ADMIN	Delete account
POST	/api/admin/accounts/{id}/deposit	ADMIN	Admin deposit
POST	/api/admin/accounts/{id}/withdraw	ADMIN	Admin withdraw
POST	/api/admin/accounts/{id}/transfer	ADMIN	Admin transfer
GET	/api/admin/users	ADMIN	List users
GET	/api/admin/users/{id}	ADMIN	Get user
PUT	/api/admin/users/{id}/activate	ADMIN	Activate user
PUT	/api/admin/users/{id}/deactivate	ADMIN	Deactivate user
PUT	/api/admin/users/{id}/role	ADMIN	Change role
GET	/api/admin/transactions	ADMIN	View all transactions
GET	/api/admin/loans	ADMIN	View all loans
POST	/api/admin/loans/{id}/approve	ADMIN	Approve loan
POST	/api/admin/loans/{id}/reject	ADMIN	Reject loan

GET	/api/admin/cards	ADMIN	View all cards
POST	/api/admin/cards/{id}/approve	ADMIN	Approve card
POST	/api/admin/cards/{id}/reject	ADMIN	Reject card
GET	/api/admin/chequebooks	ADMIN	View all chequebooks
POST	/api/admin/chequebooks/{id}/approve	ADMIN	Approve chequebook
POST	/api/admin/chequebooks/{id}/reject	ADMIN	Reject chequebook

A.2 Validation Rules

Field	Rule
username	Required, unique
password	Required, min 8 characters
email	Required, valid email format
amount	Required, must be > 0
accountType	SAVINGS, CHECKING, FIXED_DEPOSIT
loanType	HOME, PERSONAL, AUTO, EDUCATION
cardType	CREDIT, DEBIT
numberOfLeaves	1-50 (default: 50)
CVV	Returned only at card creation

A.3 Limits

Resource	Limit
Cards per account	3
Chequebook leaves	Max 50
Card validity	3 years
Card daily limit	$500 (default)

Appendix B: Example Integration Flow

B.1 Complete User Flow

```
1. Register → Login → Get Token
2. Open Account → Receive accountId
```

```
3. Deposit Money → Verify balance
4. Apply for Card → Save CVV (important!)
5. View Card → Check status (Pending)
6. (Admin approves card)
7. View Card → Status is now Active
8. Transfer Money → Verify transaction
9. View Transactions → Confirm transfer
```

B.2 Complete Admin Flow

```
1. Login as Admin
2. View all pending requests (loans, cards, chequebooks)
3. Review and approve/reject requests
4. Manage user accounts (activate/deactivate)
5. View system-wide transactions
```

Document Version: 1.0
Last Updated: 2024
For Frontend Developers